

A loft web application, end to end

Contents

0.1. Two constraints that shape everything below	3
0.2. The invariant	3
0.3. What already exists	4
0.4. What is missing	5
0.5. The libraries	5
0.6. Rotation	8
0.7. Authentication	9
0.8. Sandbox — and exactly what it does and does not promise	9
0.9. The two examples	10
0.10. Verification — on rails that already exist	13
0.11. Its own PDF	15
0.12. Build order	15
0.13. Open questions	16

The concrete realisation of BROADENING.md § Better PHP, the 2026-08 cycle theme. Two programs a reader can run: a **client** that calls a web service with JSON and authentication, and its **counterpart** — an HTTPS server with Let's Encrypt certificates that rotate themselves, authentication, a SQL backend, and two pages, one JSON and one HTML. Plus the libraries those two programs need and do not have. **Plan: @PLN148.** Written as a design, not a plan: It states one invariant, counts what has to re-state it, lists how it breaks, and pins each claim to a check — design-protocol. The build order is at the end, deliberately last: the order is a consequence of the design, not the design.

0.1. Two constraints that shape everything below

1. We are showing, not converting. A reader must be able to take one half and leave the other. The client has to talk to *any* web service — Stripe, GitHub, an internal API written in Java — and the server has to serve *any* client — curl, a browser, a Python script. No step may be reachable only from the other half. Where a design choice would be nicer if both ends were loft, the interoperable choice wins, and where loft *does* get something extra from both ends being loft, that is a bonus paragraph, never the main path.

2. The setup is the product. “Better PHP” is not a claim about the language. PHP won the web because index.php in a directory *was the deployment*: no build, no wiring, no scaffolding. Anything in this design that makes a reader configure something before their first response is a defect in the design, not a step in the tutorial. That is the standard the rest of this document is measured against.

0.2. The invariant

Every fact that differs between a laptop and production is declared once, in `loft.toml`, and never appears in the program. A handler is a function from a typed request to a typed response.

The port, the certificate, the database, the auth realm, the plugin policy: all of it is deployment, none of it is program. A handler that names a port has already failed the invariant, because the same handler has to run on both machines.

Why this is the load-bearing claim and not just a preference. It is what makes the “near-zero ceremony” goal *testable*. Ceremony is not a feeling; it is the number of facts a reader must supply before their first response, and where they must supply them. If every deployment fact has exactly one declared home, that number is countable, and the tutorial can be written against it. If deployment facts leak into handlers, the number is unbounded and every new feature adds ceremony that no one budgeted for.

And it is what makes loft better than PHP rather than a copy. PHP moves deployment out of the script too — into httpd.conf, php.ini, and a .htaccess file, none of which the program can see and none of which is checked against it. Here the declaration is typed, it is read by the same loader that reads the program, and a mismatch between the two is a load error rather than a 500 at 3am.

0.2.1. Counting the re-assertion sites

Design-protocol step 2: how many independent places must restate the invariant, and is omitting it silent?

fact	sites that must know it	omission is
the port	1 — <code>app.run()</code>	loud (nothing binds)
the certificate	1 — the TLS resolver, <i>if</i> the cert can be swapped behind it	see below
the database handle	1 — the connection the framework hands each handler	loud (no handle)

fact	sites that must know it	omission is
the auth realm	1 — the session verifier	silent, and this is the dangerous one
the sandbox policy	1 — load-time admission	loud (admission refuses)

Two of these need work to actually be 1, and both are called out below:

- **The certificate is $N = 1$ only if the server can swap it.** Today it cannot: `server::listen_tls(port, cert, key)` builds a `rustls::ServerConfig` once at bind (`native/src/lib.rs::server_config_from_pem`). With no swap, “the cert is current” is re-asserted by *every restart*, the renewal has to be a process manager’s job, and the rotation story becomes an operations runbook instead of a library. This is the one genuine new capability the server library needs, and § *Rotation* is about earning $N = 1$.
- **The auth realm’s omission is silent, which outranks its count.** A handler that forgets to check a session does not fail to compile and does not return an error; it serves the page to everybody. $N \times$ silence is what the design has to drive down (design-protocol step 2), and a count of 1 does not help when the failure is quiet. § *Authentication* makes the omission loud instead: the route table carries the requirement, so an unprotected route is a visible row rather than a missing line inside a function.

0.3. What already exists

Measured against the tree and the catalogue on 2026-08-29, not assumed.

piece	state	where
JSON	✓ in the <code>stdlib</code> — a typed <code>JsonValue</code> tree, parse, build, render, and both struct directions (<code>struct_to_json</code> , <code>struct_from_jsonvalue</code>)	<code>default/06_json.loft</code> (@F42)
HTTP client	✓ <code>web v0.3.5</code> — GET/POST/PUT/DELETE, custom headers, range requests	use <code>web</code> ;
HTTP + WebSocket server	✓ <code>server v0.6.0</code> — routing-free request loop, typed responses, <code>respond_html</code> / <code>respond_typed</code> , CORS, static ranges	use <code>server</code> ;
HTTPS	✓ <code>server::listen_tls(port, cert, key)</code> — <code>rustls</code> , PEM chain + key	same
Crypto for ACME	✓ <code>crypto v0.3.8</code> — ECDSA P-256 (ACME’s ES256), SHA-256, HMAC, base64url , CSPRNG	use <code>crypto</code> ;
SQL	⚠ works, unpublished — one uniform interface drives four C libraries (<code>sqlite</code> , <code>MariaDB</code> , <code>PostgreSQL</code> , <code>DuckDB</code>) over the <code>#c</code> tier, and a <code>loft</code> struct has a canonical SQL shape	<code>tests/fixtures/sqlldb/</code> , @PLN23 (closed on the claim, not on a package), @PLN24
Reflection for an ORM	✓ <code>s#fields</code> (compile-time, typed payloads) and <code>type_of</code> +	<code>STDLIB.md</code>

piece	state	where
	field_value (runtime, per-field data)	
HTML escaping	✓ html v0.1.0 — escape_html	use html;
Script admission	✓ built — [sandbox] in loft.toml, proven at load: capability, termination, data integrity, backend	@PLN86, src/sandbox.rs, SANDBOX.md

The honest summary: **most of the hard parts are done**. JSON is in the language, TLS terminates, the crypto for ACME is published, SQL runs against four engines, and untrusted scripts already have a load-time proof. What is missing is not depth — it is the last mile that turns them into something a reader can use in ten minutes.

0.4. What is missing

gap	why it blocks the two examples
Certificates from Let's Encrypt	nothing speaks ACME. A reader must obtain a cert by hand, out of band, with another tool — the single biggest step between “I wrote a handler” and “it is on the internet”
Certificate rotation	listen_tls freezes the cert at bind, so renewal means a restart
Authentication	nothing hashes a password, signs a session cookie, or verifies a bearer token. Both halves need it: the client to <i>send</i> credentials, the server to <i>check</i> them
A published SQL package	@PLN23's work lives in test fixtures. use sql; does not resolve
Routing and bootstrap	server hands you a request; matching a path and a method is the reader's if chain. This is where the ceremony currently is
A per-request budget	admission proves a script <i>terminates</i> ; it does not bound how long. § <i>Sandbox</i>

0.5. The libraries

Four libraries and two changes to existing ones. They are separate on purpose — the over-unification guard (design-protocol step 4) applied to the tempting claim “*one webapp library absorbs all of it*”. That claim is false in a way that shows up immediately: acme is useful to someone with no web framework, auth is useful in a CLI that stores passwords, and sql is useful in a batch job that never serves a request. Fusing them would force three unrelated dependency sets on every consumer of any one of them.

0.5.1. What these libraries get for free

Reuse is not only a testing rule. Each of the four starts with substrate that already exists, and the design is mostly *assembly*:

new piece	stands on
acme	web (HTTP), crypto (ES256, SHA-256, base64url, CSPRNG), stdlib JSON — no new primitives
auth	crypto (HMAC, SHA-256, base64) plus two additions to it
sql	@PLN23's four-driver interface and the #c tier (@PLN24) — packaging, not building

new piece	stands on
webapp	server for the socket loop, the stdlib for JSON, html for escaping
the plugin story	@PLN86 admission, and PLACEMENT.md if an operator wants a plugin in another process or another machine — one manifest line, consumers unchanged
a new lint (e.g. “ <i>this route has no .auth()</i> ”)	the diagnostic index and --explain — a code is a frozen public surface and lands with its row (DIAGNOSTICS.md)
debugging a live server	loft debug prog.loft:12 and loft debug --serve — stop in a handler, read the live frame, step. Reach for it instead of adding println

The **PLACEMENT.md** row deserves emphasis, because it changes what the sandbox chapter can offer. An operator who does not want to rely on admission alone can move a plugin into a worker process with a manifest line and no code change. That is a second, coarser containment tier the reader can choose *without* the design having to build one — which is exactly the “people can judge for themselves” posture: two mechanisms, both declarative, both legible, and the operator picks.

0.5.2. acme — certificates from Let’s Encrypt

An ACME (RFC 8555) client: create or load an account key, place an order for a domain, answer the challenge, submit a CSR, collect the certificate, and say when to renew.

It is a **pure-loft library over what already exists** — web for HTTP, crypto for ES256 and base64url, the stdlib for JSON — with one exception: a CSR is a DER-encoded PKCS#10 structure, and DER is a byte format, not a text one. That is a small, self-contained encoder, and it belongs beside the CBOR encoder in the encoding category rather than inside acme.

```
pub fn account(dir_url: text, email: text, cache: text) -> Account
pub fn order(self: Account, domains: vector<text>) -> Order
pub fn challenge(self: Order) -> Challenge // token + key authorisation
pub fn finalize(self: Order, key: text) -> Certificate
pub fn renew_after(self: Certificate) -> integer // seconds, from notAfter
```

The one decision in it. challenge() returns the token and the expected response rather than serving it. Serving it is the framework’s job (§ *Rotation*), and a library that both computes and serves cannot be used by someone who terminates TLS elsewhere — which violates constraint 1.

0.5.3. auth — credentials, on both sides

for the client	for the server
attach basic / bearer credentials to a request	verify a bearer token
cache an OAuth2 client-credentials token until it expires	hash and check a password
	sign, issue and verify a session cookie
	issue and check a CSRF token

Password hashing is **PBKDF2-HMAC-SHA256**, and it is an addition to the crypto library rather than a loop in auth: the whole point of a password hash is that it is slow, and a hundred thousand HMAC rounds driven from loft would be slow in the wrong place — slow per *call*, not slow per *guess*, which is a denial-of-service surface rather than a defence.

Constant-time comparison is likewise a crypto primitive. An auth written over `==` for token equality is a timing oracle, and that is exactly the kind of quiet mistake a library exists to make unavailable.

0.5.4. sql — publish what @PLN23 built

The interface exists and drives sqlite, MariaDB, PostgreSQL and DuckDB unchanged. The work is packaging: a public API, a driver-selection story keyed on the connection string (there is already one — `one_connection_string_reaches_its_driver_and_a_refusal_behaves_like_one` in `tests/native.rs`), and the items @PLN23's closure record listed as *not yet shipping-grade*: data enums, tuples, concurrency, and reading a schema back.

The one design rule the package must enforce: a query takes parameters, and building a query by concatenation must be harder than not. SQL injection is PHP's most famous wound, and it is a wound of *ergonomics*, not of ignorance — concatenation was the shortest path. Here the shortest path is `q.param(name)`, and the escape hatch for what parameters cannot express is spelled out loudly, exactly as `store_lazy_query` already does for the derived-query case in the `stdlib`.

0.5.5. webapp — the ceremony sink

This is the library the whole design is for. It is thin: it owns the route table, the bootstrap that reads `loft.toml`, and nothing else that the other three do.

```
use webapp;

fn main() {
    app = webapp::app();
    app.get("/", home);           // HTML page
    app.get("/api/notes", list_notes); // JSON page
    app.post("/api/notes", add_note).auth(); // requires a session
    app.run();
}
```

and the deployment, in `loft.toml`:

```
[web]
domain = "notes.example.com"
port   = 443

[web.tls]
acme = "letsencrypt"           # or "staging", or a cert/key pair by path
email = "ops@example.com"
cache = "/var/lib/loft/acme"   # account key + certificates live here

[web.db]
url = "sqlite:notes.db"        # or postgres://... — the driver follows the scheme

[web.auth]
sessions = "cookie"
secret   = "env:NOTES_SESSION_SECRET" # never a literal
```

`app.run()` is the whole deployment story, and every line above is a fact the program never mentions. Running the same program on a laptop means one edit — `port = 8080` and no `[web.tls]` — and the handlers do not change. That is the invariant, made concrete.

`.auth()` is the answer to the silent-omission problem. The requirement sits in the route table, where a reader sees the protected and unprotected routes as adjacent lines and an unprotected one is a visible absence. A check buried inside a handler body is invisible from the outside — you have to open every handler to learn

which pages are public, and forgetting one produces no signal at all. The route table also lets the framework *report* the split: `loft` routes printing which paths are open is a diagnostic that a per-handler check cannot offer.

0.5.6. server — one change: `reload_tls`

```
pub fn reload_tls(self: Server, cert: text, key: text) -> boolean
```

`rustls` already supports this shape: the listener holds a certificate resolver, and swapping what the resolver returns changes the certificate for the *next* handshake while every in-flight connection keeps the one it negotiated. Without it, $N = 1$ for the certificate is not reachable and rotation is not a library's business.

0.5.7. encoding — one addition: DER

A minimal DER writer, enough for PKCS#10. Its home is the encoding category beside `cbor`, not `acme`, because certificate signing requests are not the only thing that wants DER.

0.6. Rotation

The protocol, and the reason each step is where it is:

1. **At start**, read the cached certificate. If it is valid and not near expiry, bind with it. If there is none, obtain one before binding.
2. **Renew at one third of the remaining lifetime**, computed from the certificate's own `notAfter` — never from a local counter. A counter is wrong across a restart and wrong across a clock change; the certificate carries the truth.
3. **Answer the challenge from inside the app**. `/.well-known/acme-challenge/<token>` is a route the framework owns and the reader never writes. This is the step other stacks push into a web-server configuration file, and it is precisely the step that makes people give up.
4. **On success, `reload_tls`**. No restart, no dropped connection.
5. **On failure, keep serving the old certificate and try again**. A failed renewal is not an outage; it becomes one only at expiry. The distinction has to be in the code, because the naive shape — `renew` or `die` — turns a rate limit into downtime.
6. **One renewer at a time**, through a lock file in the cache directory. Two processes sharing a domain will otherwise both order, and Let's Encrypt's rate limits are per domain.

The failure paths, written down before the code — this enumeration is what surfaces the requirements, and is the reason this section exists before any of it is built:

what goes wrong	what must happen	what must <i>not</i> happen
ACME is rate-limited	keep the old cert, retry with backoff	exit, or bind with nothing
the order fails validation	log loudly, keep serving, retry	serve a half-provisioned cert
the process restarts	reuse the cached cert <i>and</i> account key	order a new cert on every restart
the cache directory is lost	order again; the account key is gone, so make a new account	fail to start
two processes renew at once	one wins the lock, the other reads the result	two orders against one domain
the clock is wrong	renewal derives from <code>notAfter</code> , so a wrong clock delays or hastens but never corrupts	a counter that survives nothing

what goes wrong	what must happen	what must <i>not</i> happen
the cert expires anyway	fail the <i>handshake</i> loudly with an alarm; the app is still up on port 80 for the challenge	serve an expired cert silently
port 80 is unreachable	the HTTP-01 challenge cannot work — say so at <i>configuration</i> time, naming DNS-01 as the alternative	discover it at renewal, months later

The last row is the one that most deserves its place. A misconfiguration that only shows up sixty days later, in the middle of the night, is the worst failure mode this design can ship, and the cure is cheap: the thing that can be checked at start-up is checked at start-up.

0.7. Authentication

Three separate questions that get confused with each other. Keeping them apart is most of the design:

question	mechanism	where
who is calling this API?	bearer token, checked in constant time	auth, server side
who is this person in a browser?	signed session cookie, HttpOnly + Secure + SameSite=Lax	auth, server side
how do I prove <i>I</i> am allowed to call <i>them</i> ?	basic / bearer / OAuth2 client credentials, with the token cached until it expires	auth, client side

Cookie flags are set by the library, not by the reader. Every one of those three flags is a well-known way to lose sessions to script injection or cross-site requests, every one is a single word, and every one is routinely forgotten. A library that accepts them as parameters has moved a security decision to the least-informed place in the system.

The password path is deliberately narrow: store a PBKDF2 hash, verify against it, and offer no way to retrieve the original. There is no API that returns a password.

0.8. Sandbox — and exactly what it does and does not promise

@PLN86 already ships the hard half. A host declares a policy, and a designated function is admitted only if it is **proven** at load to satisfy four arcs: capability (it reaches only the libraries it was granted), termination (bounded loops, acyclic recursion, a total op bound), data integrity (no raw writes to host data; per-field read/update/append rights), and backend (no external FFI). It is refused with an actionable error otherwise.

For a web application the profile is a plugin directory:

```
[sandbox]
plugins = ["plugins/*.loft"]

[profile.plugins]
allow_libs = ["code", "text"]    # no net, no sql, no files
```

An admitted plugin gets a request's *data*, and can compute with it at full interpreter speed. It cannot open a socket, cannot read the database, and cannot write host state, because it never had those capabilities in the first place — not because something stopped it at runtime.

0.8.1. Where the line actually is — and the one thing a web host needs that a game did not

This is the design’s cleanest-looking claim, so it is the one to attack (design-protocol step 4). The tempting sentence is “@PLN86 already solves untrusted third-party scripts”. It does not, and the difference matters more on a web server than in the game it was designed for.

@PLN86 is compile-only by an explicit decision: there are no runtime checks. For a game that is right — a frame boundary is not a place you can afford a budget check, and a mod that is bounded is good enough. A web request is a different context in two ways:

- **Bounded is not fast.** The termination arc proves a loop *ends*. It does not prove it ends in fifty milliseconds. A plugin that is entirely legitimate under admission can still hold a request thread for a long time, and on a server that is an availability problem rather than a stutter.
- **A request boundary is affordable.** The reason the game dropped runtime checks does not apply. Requests are already the unit of work, already have a deadline, and already have a timeout the caller can see.

So the web profile adds **one runtime bound the game deliberately did not have**: a per-request op and wall-clock budget, enforced at the request boundary, that aborts the *plugin* and returns an error for that request. Not a new isolation mechanism — the substrate is the same op counter the profiler already rides (LOFT_PROFILE, PERFORMANCE.md) — but it is a real extension of @PLN86’s scope and it is recorded here as one rather than left implied.

And what remains uncovered, stated plainly, because the operator is the one judging. The user’s framing is the right one: *people can judge for themselves whether they want to allow third-party scripts*. That judgement is only possible if the guarantee is legible. So:

covered by admission	not covered
reaches only granted libraries	anything a granted library can do — grant net and the script can call out
terminates	how much heap it touches beyond the load-time data envelope
no raw writes to host data	logical mistakes inside the rights it was granted
no FFI	the correctness of the policy you wrote

The last row is the important one and belongs in the tutorial, not in a footnote: **the policy is the security boundary, and it is the operator’s to write**. The design’s job is to make it short, declarative, checked at load, and printable — so that “what can this plugin do?” is a question with an answer, rather than a code review.

0.9. The two examples

Both are ordinary loft programs a reader can copy, and both are executed by the test suite.

0.9.1. What runs today, measured

Before designing what is missing, here is the pair built **only from what already ships** — no webapp, no auth, no sql. It was run, and the output below is the real transcript, not an illustration.

srv.loft:

```
use server;

struct Note { id: integer, title: text }
struct Notes { notes: vector<Note> }

fn main() {
  srv = server::listen(18711);
  for _ in 0..2 {
```

```

req = srv.next();
if req.path == "/api/notes" {
    body = Notes { notes: [Note { id: 1, title: "first" },
                           Note { id: 2, title: "second" }] };
    req.respond_typed(200, body.to_json(), "application/json");
} else {
    req.respond_html("<h1>Notes</h1><ul><li>first</li><li>second</li></ul>");
}
}
srv.close();
}

```

cli.loft:

```

use web;

fn main() {
    r = web::http_get_h("http://127.0.0.1:18711/api/notes",
                       ["Authorization: Bearer demo-token"]);
    if !r.ok() { println("request failed: {r.status}"); return; }
    doc = json_parse(r.body);
    notes = doc.field("notes");
    for i in 0..notes.len() {
        n = notes.item(i);
        println("{n.field(\"id\").as_long(): {n.field(\"title\").as_text()}}");
    }
}

```

and the run:

```

$ loft srv.loft &
$ loft cli.loft
1: first
2: second
$ curl -s http://127.0.0.1:18711/
<h1>Notes</h1><ul><li>first</li><li>second</li></ul>

```

Run three ways, because a pass on one backend is not a pass:

how	result
--interpret, both sides	1: first / 2: second, and curl gets the HTML
--native, both sides	identical
LOFT_STRICT_STORES=1, both sides	clean — no store outlives the run

The strict-stores row is the one that would not have been checked by hand. A request handler is a loop body that runs a million times, so a per-request leak is the defect this stack is most likely to ship, and it is invisible to a functional test that serves one request. The existing gate answers it for free.

Three things this measurement settled, which reading the catalogue would not have.

1. **The struct is the JSON.** `body.to_json()` on an ordinary struct produced the wire format, and `json_parse + field` read it back on the other side. No serialisation layer, no annotations, no code generation. This is the strongest single argument the guide has, and it needed no new library.

2. **Hand-written JSON in a string literal is hostile, and the design must never show it.** `{` opens a format-interpolation hole, so a literal `"{\"id\":1}"` needs every brace doubled and otherwise produces a wall of parse errors. The first attempt at this probe hit exactly that. The cure is the idiom above, so the rule for the guide is: **a handler builds JSON from a struct or with the `stdlib` constructors, never as text.** That is not style advice; it is a consequence of the language's format strings, and stating it once is worth more than a reader discovering it through fourteen errors.
3. **`curl` got the HTML.** Constraint 1 is not an aspiration — the server is already an ordinary HTTP server that anything can call.

The probe also found a defect and it is filed: the library prints `loft server listening on 0.0.0.0:18711 (http/ws)` on `stderr`, unrequested (`loft#1174`). The first page of the guide would have printed it and had to explain it.

0.9.2. What the design adds

The client, once `auth` exists — the only line that changes is the first:

```
use web;
use auth;

fn main() {
    token = auth::bearer(env_variable("API_TOKEN"));
    r = web::http_get_h("https://api.example.com/notes", token.headers());
    ...
}
```

The server, once `webapp` and `sql` exist:

```
use webapp;
use sql;

struct Note { id: integer, title: text, body: text }

fn main() {
    app = webapp::app();
    app.get("/", home);
    app.get("/api/notes", api_notes);
    app.post("/api/notes", add_note).auth();
    app.run();
}

fn api_notes(r: Request) -> Response {
    notes = r.db.query("select id, title, body from note order by id").rows(Note);
    return r.json(notes);
}

fn home(r: Request) -> Response {
    rows = "";
    for n in r.db.query("select id, title from note order by id").rows(Note) {
        rows += "<li>{escape_html(n.title)}</li>";
    }
    return r.html("<h1>Notes</h1><ul>{rows}</ul>");
}
```

Read what is absent. No port, no certificate, no ACME, no connection string, no cookie flags, no renewal timer, and no accept loop. That absence is the invariant holding, and it is the only honest way to demonstrate “better

PHP” — not by asserting low ceremony, but by showing a program with none in it. The measured version above is what the same program costs today: an accept loop, a hand-written route branch, a literal port, and no HTTPS at all.

0.10. Verification — on rails that already exist

The design rule: no new harness unless a named existing instrument cannot answer the question. A web stack is exactly the kind of work that grows a bespoke test rig per library — a mock HTTP server here, a fake clock there, a shell script that curls a port — and five rigs are five things that rot. Everything below is an instrument the repo already has, with the one place a new one is genuinely needed called out as such.

0.10.1. The examples themselves

question	instrument	already used by
does every example compile and run?	tests/docs/ topic harness	the 38 language-reference topics
...on both backends?	features_examples_interpret + native_features	the @F catalogue
...including one that says use webapp?	run_via_loft_binary — the delegation added for loft#1173, which builds a library-importing example through the real loft --native	@F43
does a library example agree across backends?	tests/doc_lib_examples.rs — subprocess through the real binary, interpret == native	14-image, 21-random
does the printed output match the page?	tests/doc_commands.rs — runs every \$ line on a page and asserts each following line appears	the four CLI pages
is a refusal the one we meant?	@EXPECT_ERROR / @EXPECT_WARNING corpus annotations	throughout tests/

The use webapp row is the load-bearing one and it is already paid for. A documentation example that imports a library could not be built under --native until this week; that gap is closed, so every page in the new document is a both-backend example from the first one.

0.10.2. The client and the server, against each other

The loopback round-trip needs no new rig. The server binds an ephemeral port under try_listen_tls, the client example runs against it, and the page’s transcript is the assertion — the same shape tests/doc_commands.rs already applies to CLI pages, with a \$ loft server.loft & line instead of a one-shot command.

Three existing instruments do the rest, and each answers a question a plain pass/fail cannot:

- **LOFT_NET_PROFILE=1 | trace** reports socket operations **by margin** — a call that finished close to its deadline is a failure that has not happened yet — with wall-clock stamps that *merge two processes’ streams*. That is built for exactly this test: one report covering the client and the server together. It records at the sockets the runtime owns, and a networking library joins by calling `loft::net_profile::time(...)` from its Rust bridge — so **web and server should join it**, which is a small change to two existing libraries rather than a new measurement system. The armed-but-empty report says so rather than printing nothing (loft#1088), so a library that has *not* joined is visible instead of silently unmeasured.

- **LOFT_STRICT_STORES=1** scores every example for leaks. A request handler is a loop body that runs a million times; a per-request leak is the defect this stack is most likely to ship, and it is invisible to a functional test that serves one request.
- **The store-memory ceiling** (LOFT_MEMORY_LIMIT, 2 GiB under test runs) bounds a runaway before it takes the box, and names the *type* that filled the heap — which for a server is usually the answer.

0.10.3. The ACME path

Against a **local ACME stub written with the server library itself** — not a new mock framework. Dogfooding the library the design is shipping is cheaper than a rig, and it makes the stub a consumer that would notice a server regression.

Renewal timing is driven by an **injected clock**, because a test that waits sixty days is not a test. That injection point is the one genuinely new piece of test infrastructure in this design, and it is one parameter on `acme::renew_after`, not a harness.

Each of the eight rows in § *Rotation* becomes a cell, hand-computed before it is run (CLAUDE.md § *matrix-first*). Two supports already exist for getting that right:

- **scripts/matrix_axes.py** answers which composition axes the cells actually reach and which they do not — derived, not declared. The axes here are real and easy to pin by accident: challenge type, cache state, clock offset, concurrent renewer, failure kind.
- **make falsify** answers whether each guard *fails* on the build it was written to catch, and names the channel that moved. The rate-limit and expired-cert rows are the two a happy path would miss, and they are the two that produce a real outage — so they are the two whose falsification matters most.

0.10.4. The sandbox

Admission is a **load-time** property, so its tests are corpus files with `@EXPECT_ERROR`: a plugin that calls `net` under a `[ "code", "text" ]` profile must be refused, by name, with the actionable error `@PLN86` already produces. No runtime rig at all.

The per-request budget is the one runtime addition, and it rides substrate that exists: the same per-op debug branch the sampler uses (LOFT_PROFILE, PERFORMANCE.md) and the same counter shape as `LOFT_MAX_OPS`. Building it on a third counter would be a fourth place that has to agree about what an operation is.

0.10.5. The libraries as shipped artefacts

Publishing has its own rails, and this design uses them rather than adding release steps:

question	instrument
does the published library still build and pass?	<code>revalidate_libs_local.sh</code> — <code>make ci</code> says nothing about the shipped libraries
what is each library's real API and where does it diverge?	<code>make libcatalogue</code> → <code>LIBRARIES.md</code> ; <code>scripts/lib-overlay.py</code> <name> for provenance
are its diagnostics clean enough to gate?	<code>LOFT_DENY_WARNINGS=1</code> , the warning tier
is a function documented, and has it moved since?	<code>make libraries-review</code> , <code>scripts/doc-review.sh --since</code>
does the catalogue know these features exist?	@F entries via <code>make features-fetch</code> && <code>make features-gen</code>
how does it get signed and published?	the loft-ship skill

0.10.6. What performance claims are allowed to say

`make speed` and `make profile` are **reports**, **never gates**. The “persistent, typed, natively-fast process” argument in *BROADENING.md* is a claim about *throughput across requests*, and the honest instrument for it is

LOFT_PROFILE with LOFT_PROFILE_EVERY or kill -USR1 — because a server’s only exit is a signal, and the run you most want a profile of is the one that cannot produce one at exit (loft#1089). The profiler notes that a used library is a cdylib it cannot enter, so a library doing the work reads as a hot caller; for this stack — where sql and server *are* the work — that note is the difference between a useful profile and a wrong one.

0.11. Its own PDF

Two documents, two pipelines, and they are not the same thing.

This design prints today. scripts/md2typ.py renders any doc/claude/ design document as Typst, and make pdf-doc DOC=doc/claude/WEB_STACK.md OUT=doc/web-stack compiles it. The Markdown stays the single source, so the page and the PDF cannot drift — the one-home rule applied to a document rather than to code.

The user guide is generated from running code, which is a different and stronger property, and it lands at stage 5. The guide is a **separate document from the language reference**. A reader deploying a web application does not want the type system, and a reader learning the language does not want ACME. Same pipeline, second output:

```
tests/docs/web/*.loft → cargo run --bin gendoc → doc/loft-web.typ → doc/loft-  
web.pdf
```

The topic files are executable loft programs carrying their prose in comments, exactly like the existing tests/docs/*.loft topics — which is what makes “every example in this document is an executable part of the test suite” true of the new document too, and not merely asserted. gendoc gains a second topic root and a second generate_typst call; the existing document is unaffected.

Planned contents:

page	what it shows
1 · Your first response	one file, loft.toml, loft run — a page in the browser
2 · Talking to a web service	web + json_parse, the client example
3 · Sending credentials	bearer, basic, OAuth2 client credentials
4 · Serving JSON	struct → JSON, status codes, error shapes
5 · Serving HTML	escaping, and why the escape is not optional
6 · The database	connection string, parameters, and why concatenation is not offered
7 · Logging people in	sessions, password storage, CSRF
8 · Going live with HTTPS	Let’s Encrypt, the challenge route, rotation
9 · Running other people’s scripts	the sandbox profile, what it guarantees, what it does not

0.12. Build order

Each stage ends with something a reader can run, because a stage that ends in a library nobody can use yet is a stage whose design has not been tested.

stage	what lands	ends with
1	sql published from @PLN23; the webapp route table and loft.toml bootstrap	pages 1, 4, 5, 6 — an HTTP app with a database
2	crypto: PBKDF2 + constant-time compare. auth both sides	pages 3, 7 — logging in, and the client example
3	server::reload_tls; encoding: DER; acme	page 8 — HTTPS that renews itself

stage	what lands	ends with
4	the web sandbox profile + the per-request budget	page 9
5	gendoc's second topic root; doc/loft-web.typ	the PDF

Stage 1 is the one that proves the invariant, and it is deliberately first for that reason. If a reader cannot get a database-backed page up with no deployment facts in their program, nothing later in the list rescues the design — and the cheapest place to discover that is stage 1, not stage 5.

0.13. Open questions

1. **Does webapp own a template file format, or is a format-string enough?** BROADENING.md left this open. The examples above use format strings and `escape_html`, which is the smaller answer; whether it survives a page with real structure is a question for stage 1, answered by writing page 5 and seeing.
2. **Is webapp one library or does routing separate from bootstrap?** They have different audiences — someone terminating TLS at a proxy wants the routes and not the bootstrap. Deferred to stage 1, where the answer is observable rather than guessed.
3. **HTTP-01 or DNS-01 first?** HTTP-01 needs port 80 and works with no credentials; DNS-01 works behind a proxy but needs a DNS provider API. HTTP-01 is the smaller first target and covers the reader this document is written for.